TF4015 – Sistem Kontrol Diskrit

Praktikum

Modul 2

Kontrol Kecepatan Motor DC

Tim Pengajar TF4015

Program Studi Teknik Fisika
Fakultas Teknologi Industri
Institut Teknologi Bandung

Semester I 2026/2027

Daftar Isi

1 Pendahuluan **1**
1.1 Latar Belakang . . . 1
1.2 Tujuan Praktikum . . . 1
1.3 Peralatan dan Perangkat Lunak . . . 1
1.4 Persiapan Praktikum . . . 2

2 Teori Dasar **2**
2.1 Sistem Kontrol Kecepatan Kalang Tertutup . . . 2
2.2 Pengontrol PID Kontinu dan Diskrit . . . 2
2.3 Saturasi dan *Anti-Windup* . . . 3
2.4 Penalaan Ziegler–Nichols dan Cohen–Coon . . . 3
2.5 Parameter Performa Pengontrolan . . . 4

3 Keselamatan Kerja **5**

4 Prosedur Praktikum **5**
4.1 Percobaan 1: Parameter Kontrol dan Simulasi . . . 5
4.2 Percobaan 2: Persiapan Program Kontrol . . . 6
4.3 Percobaan 3: Implementasi Kontrol Kecepatan . . . 6
4.4 Percobaan 4: Penalaan Halus . . . 7

5 Laporan dan Analisis **8**
5.1 Ketentuan Laporan . . . 8
5.2 Analisis . . . 8

6 Tugas Tambahan **9**

A Kode Program Pendukung **11**

1 Pendahuluan

1.1 Latar Belakang

Pengendalian kecepatan putar motor merupakan kasus inti pada berbagai aplikasi, seperti elevator, ban berjalan, dan pompa. Pada Modul 1, kecepatan motor DC telah dimodelkan sebagai sistem orde satu dengan waktu tunda (FOPDT), *deadzone* telah dikompensasi, dan waktu cacah telah ditentukan. Dengan bekal tersebut, pada Modul 2 sistem dikendalikan secara **kalang tertutup**.

Parameter pengontrol PID dihitung dari model FOPDT menggunakan metode empirik Ziegler–Nichols dan Cohen–Coon, lalu disimulasikan sebelum diterapkan pada kit. Pengontrol diimplementasikan sebagai PID diskrit pada Arduino, lengkap dengan pembatas keluaran dan *anti-windup*. Performa hasil simulasi dan implementasi dibandingkan, kemudian ditingkatkan melalui penalaan halus. Praktikan juga mengamati pengaruh waktu cacah terhadap performa pengontrolan.

1.2 Tujuan Praktikum

Setelah menyelesaikan Modul 2, praktikan diharapkan mampu:

1. Menghitung parameter pengontrol P, PI, dan PID dengan metode Ziegler–Nichols dan Cohen–Coon, lalu menyimulasikannya.
2. Mengimplementasikan pengontrol PID diskrit dengan pembatas keluaran dan *anti-windup* pada kit praktikum.
3. Menilai performa pengontrolan secara kualitatif dan kuantitatif.
4. Meningkatkan performa pengontrolan melalui penalaan halus.
5. Menganalisis pengaruh waktu cacah terhadap performa pengontrolan.

1.3 Peralatan dan Perangkat Lunak

Tabel 1: Daftar peralatan praktikum Modul 2

No	Nama Peralatan	Keterangan
1	Kit praktikum SKD (kit yang sama dengan Modul 0 dan 1)	Disediakan asisten
2	Laptop dengan Arduino IDE dan *library* Adafruit MCP4725	Milik praktikan
3	MATLAB (desktop atau MATLAB Online)	Milik praktikan
4	Perangkat lunak *spreadsheet*	Milik praktikan
5	Program: `5_control.ino` (versi Modul 2), `8_main_kontrol_kecepatan`	Unduh dari repositori
6	Skrip MATLAB `simulasi_kontrol_kecepatan.m` dan `identifikasi_fopdt.m`	Unduh dari repositori
7	Model final dan kompensasi *deadzone* dari Modul 1	Hasil praktikum sebelumnya

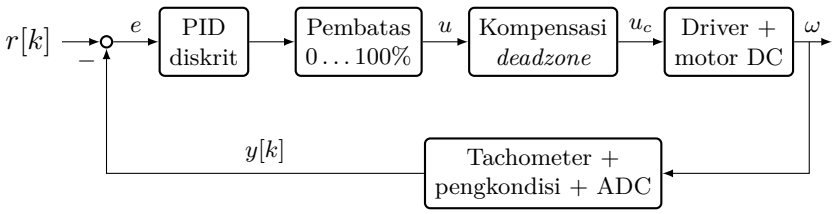
1.4 Persiapan Praktikum

1. Siapkan model final Modul 1 (K, τ, θ, T_{s1}, T_{s2}, kecepatan maksimum) dan nilai `DeadZonePositif` serta `DeadZoneNegatif` yang digunakan.
2. Ganti tab `5_control.ino` dengan versi Modul 2 (Kode 1) dan pahami setiap barisnya.
3. Jalankan skrip `simulasi_kontrol_kecepatan.m` dengan model kelompok. Hasil simulasi sebaiknya sudah tersedia sebelum praktikum.
4. Baca prosedur dan bagian analisis, lalu bagi tugas dalam kelompok.

2 Teori Dasar

2.1 Sistem Kontrol Kecepatan Kalang Tertutup

Struktur sistem kontrol kecepatan pada kit praktikum ditunjukkan pada Gambar 1. Set point kecepatan r dibandingkan dengan kecepatan terukur y untuk menghasilkan error $e = r - y$. Pengontrol PID menghitung aksi kontrol u yang kemudian dibatasi, dikompensasi *deadzone*-nya, dan diberikan kepada driver motor. Kecepatan motor diukur oleh tachometer melalui rangkaian pengkondisi sinyal dan ADC. Seluruh perhitungan dilakukan setiap waktu cacah T_s.



Gambar 1: Diagram blok sistem kontrol kecepatan pada kit praktikum

Model yang digunakan untuk perancangan adalah model FOPDT dari Modul 1, yaitu hubungan antara masukan sebelum kompensasi u dan kecepatan terukur y:

$$G(s) = \frac{K\, e^{-\theta s}}{\tau s + 1}. \tag{1}$$

Karena tachometer hanya membaca besar kecepatan, kontrol kecepatan pada modul ini dibatasi untuk satu arah putar, dan keluaran pengontrol dibatasi pada $0 \le u \le 100\%$.

2.2 Pengontrol PID Kontinu dan Diskrit

Pengontrol PID kontinu dinyatakan sebagai

$$u(t) = K_p \left(e(t) + \frac{1}{T_i} \int_0^t e(\lambda)\, d\lambda + T_d \frac{de(t)}{dt} \right) = K_p\, e(t) + K_i \int_0^t e(\lambda)\, d\lambda + K_d \frac{de(t)}{dt}, \tag{2}$$

dengan $K_i = K_p/T_i$ dan $K_d = K_p T_d$. Pada mikrokontroler, integral didekati dengan penjumlahan persegi panjang dan turunan didekati dengan selisih mundur, sehingga diperoleh PID diskrit bentuk posisi

$$u[k] = K_p\, e[k] + K_i\, T_s \sum_{i=0}^{k} e[i] + K_d \frac{e[k] - e[k-1]}{T_s}. \tag{3}$$

Pada tab `5_control.ino`, waktu dinyatakan dalam milidetik, sehingga K_i bersatuan 1/ms dan K_d bersatuan ms. Dengan cara ini, nilai K_p, K_i, dan K_d **tidak bergantung pada waktu cacah**; waktu cacah hanya memengaruhi seberapa baik persamaan (3) mendekati persamaan (2).

Selain itu, pencacahan dengan *zero-order hold* menambah keterlambatan efektif sekitar setengah waktu cacah. Sistem yang dikontrol secara diskrit berperilaku kurang lebih seperti sistem kontinu dengan waktu tunda

$$\theta_{\text{efektif}} \approx \theta + \frac{T_s}{2}. \tag{4}$$

Semakin besar T_s, semakin besar waktu tunda efektif, sehingga margin kestabilan berkurang.

2.3 Saturasi dan *Anti-Windup*

Aksi kontrol pada kit dibatasi pada 0–100%. Ketika keluaran saturasi, misalnya saat set point berubah besar, error tetap diakumulasi oleh bagian integral sehingga nilainya membesar jauh melebihi yang dibutuhkan (*integrator windup*). Akibatnya, setelah keluaran mencapai set point, integral memerlukan waktu lama untuk turun dan menyebabkan *overshoot* besar. Tab `5_control.ino` menggunakan **integrasi bersyarat**: error hanya diakumulasi bila keluaran tidak saturasi, atau bila tanda error membantu keluaran keluar dari saturasi.

2.4 Penalaan Ziegler–Nichols dan Cohen–Coon

Untuk model FOPDT, parameter awal pengontrol dapat dihitung dengan rumus empirik pada Tabel 2 [1]. Metode Ziegler–Nichols kurva reaksi umumnya menghasilkan respons dengan rasio peluruhan seperempat, sedangkan Cohen–Coon memperhitungkan rasio $r = \theta/\tau$ secara lebih eksplisit.

Tabel 2: Rumus penalaan untuk model FOPDT ($r = \theta/\tau$)

Metode	**Kontrol**	K_p	T_i, T_d
Ziegler–Nichols	P	$\dfrac{\tau}{K\theta}$	–
	PI	$\dfrac{0{,}9\,\tau}{K\theta}$	$T_i = 3{,}33\,\theta$
	PID	$\dfrac{1{,}2\,\tau}{K\theta}$	$T_i = 2\theta$, $T_d = 0{,}5\theta$
Cohen–Coon	P	$\dfrac{\tau}{K\theta}\left(1 + \dfrac{r}{3}\right)$	–
	PI	$\dfrac{\tau}{K\theta}\left(0{,}9 + \dfrac{r}{12}\right)$	$T_i = \theta\,\dfrac{30 + 3r}{9 + 20r}$
	PID	$\dfrac{\tau}{K\theta}\left(\dfrac{4}{3} + \dfrac{r}{4}\right)$	$T_i = \theta\,\dfrac{32 + 6r}{13 + 8r}$, $T_d = \dfrac{4\theta}{11 + 2r}$

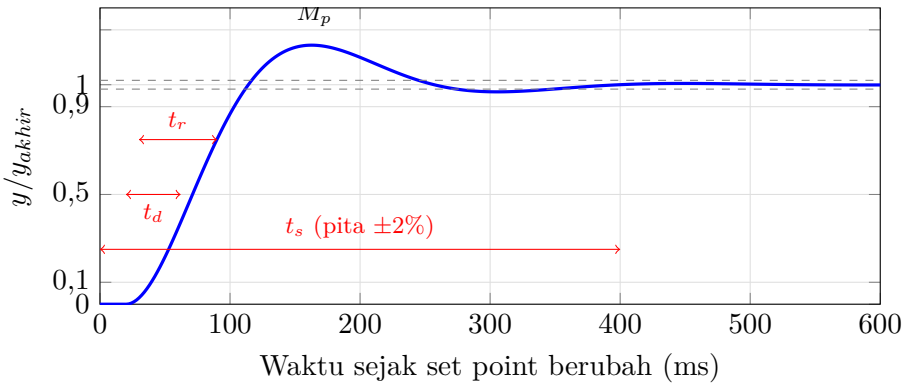
2.5 Parameter Performa Pengontrolan

Performa respons terhadap perubahan set point dinilai dengan parameter berikut (Gambar 2):

- ***Delay time*** t_d: waktu hingga respons mencapai 50% nilai akhir.
- ***Rise time*** t_r: waktu yang diperlukan respons untuk naik dari 10% hingga 90% nilai akhir.
- ***Overshoot*** M_p: selisih nilai puncak terhadap nilai akhir, dalam persen nilai akhir.
- ***Settling time*** t_s: waktu hingga respons tetap berada dalam pita ±2% nilai akhir.
- ***Offset*** e_{ss}: selisih set point dengan nilai akhir respons.

Semua waktu dihitung sejak set point berubah. Untuk pengontrol P pada sistem FOPDT, nilai akhir tidak sama dengan set point; offset teoretisnya adalah

$$e_{ss} = \frac{r}{1 + K K_p}. \tag{5}$$



Gambar 2: Ilustrasi parameter performa respons terhadap perubahan set point

Pengaruh umum setiap parameter terhadap respons kalang tertutup dirangkum pada Tabel 3. Tabel ini menjadi panduan kualitatif pada penalaan halus; pengaruh sebenarnya dapat berbeda karena saling bergantung.

Tabel 3: Pengaruh umum kenaikan parameter PID

Parameter naik	***Rise time***	***Overshoot***	***Settling time***	**Offset**
K_p	turun	naik	sedikit berubah	turun
K_i	turun	naik	naik	hilang
K_d	sedikit berubah	turun	turun	tidak berubah

3 Keselamatan Kerja

Seluruh ketentuan keselamatan kerja dan SOP pada Modul 0 tetap berlaku. Perhatikan pula hal-hal berikut:

1. Verifikasi setiap perubahan program tanpa daya motor. Pastikan kolom `MV` tidak pernah negatif dan tidak melebihi 100.
2. Bila motor berosilasi keras atau kecepatan tidak terkendali, segera cabut kabel listrik 220 V, lalu turunkan K_p atau K_d.
3. Jangan mengubah trimpot penguatan OpAmp maupun nilai kompensasi *deadzone*, karena model Modul 1 bergantung pada keduanya.

4 Prosedur Praktikum

Prosedur Modul 2 terdiri atas empat percobaan. Percobaan 1 dikerjakan di MATLAB dan dapat dilakukan secara paralel dengan persiapan program pada Percobaan 2. Simpan seluruh data mentah dari *Serial Monitor* ke file teks.

4.1 Percobaan 1: Parameter Kontrol dan Simulasi

1. Lengkapi Tabel 4 dengan model final dari Modul 1.
2. Isi parameter model pada skrip `simulasi_kontrol_kecepatan.m` (Kode 3). Atur `SetPoint` sekitar 75% kecepatan maksimum, lalu jalankan skrip.
3. Lengkapi Tabel 5 dengan parameter yang dihitung skrip. Periksa minimal satu baris secara manual dengan Tabel 2.
4. Lengkapi Tabel 6 dengan performa hasil simulasi untuk kedua waktu cacah. Simpan grafik respons.
5. Sebagai pembanding, gunakan program `8_main_kontrol_kecepatan` dengan `ClosedLoop = false` dan `SetPointSequence = {0, SP, 0}` untuk mengambil respons kalang terbuka kit dengan masukan sama dengan set point simulasi.

Tabel 4: Model sistem kecepatan kit praktikum (dari Modul 1)

No	Parameter	Nilai	Satuan
1	Penguatan K		–
2	Konstanta waktu τ		ms
3	Waktu tunda θ		ms
4	Kecepatan maksimum		%
5	Waktu cacah pertama T_{s1}		ms
6	Waktu cacah kedua T_{s2}		ms

Tabel 5: Parameter kontrol Ziegler–Nichols dan Cohen–Coon

Metode	**Kontrol**	K_p	T_i **(ms)**	T_d **(ms)**	K_i **(1/ms)**	K_d **(ms)**
ZN	P		–	–	–	–
	PI			–		–
	PID					
CC	P		–	–	–	–
	PI			–		–
	PID					

Tabel 6: Performa simulasi kontrol kecepatan

T_s	**Metode**	**Kontrol**	t_d **(ms)**	t_r **(ms)**	M_p **(%)**	t_s **(ms)**	**Offset (%)**
T_{s1}	ZN	P					
		PI					
		PID					
	CC	P					
		PI					
		PID					
T_{s2}	ZN	P					
		PI					
		PID					
	CC	P					
		PI					
		PID					

4.2 Percobaan 2: Persiapan Program Kontrol

1. Pastikan tab `4_compensation.ino` berisi kompensasi dari Modul 1 dan tab `5_control.ino` berisi versi Modul 2.
2. Ganti tab `9_main.ino` dengan program `8_main_kontrol_kecepatan` (Kode 2). Atur `TimeSampling` $= T_{s1}$, parameter pengontrol P dari Ziegler–Nichols, dan `SetPointSequence = {0, SP, 0}`.
3. Tanpa daya motor, jalankan program dan periksa bahwa kolom `MV` bernilai $K_p\times$ set point pada awal step (karena kecepatan nol), bahwa kolom `MV` tidak melebihi 100, dan bahwa kolom `Motor` sesuai fungsi kompensasi.
4. Tanpa daya motor, uji pula pengontrol PI. Amati bahwa `MV` naik hingga 100 lalu tertahan (saturasi). Jelaskan apa yang terjadi pada variabel integral bila *anti-windup* tidak digunakan.

4.3 Percobaan 3: Implementasi Kontrol Kecepatan

1. Hubungkan kabel listrik 220 V. Jalankan pengontrol P, PI, dan PID untuk metode Ziegler–Nichols dan Cohen–Coon dengan urutan set point `{0, SP, 0}` pada T_{s1}. Simpan setiap data respons.

2. Ulangi untuk T_{s2}.
3. Hitung parameter performa setiap respons (gunakan bagian fungsi `kinerja` pada skrip simulasi atau *spreadsheet*) dan lengkapi Tabel 7 dengan format yang sama dengan Tabel 6.
4. Pilih satu pengontrol terbaik untuk setiap waktu cacah. Jalankan dengan urutan set point `{0, 50, 80, 20, 0}` untuk menguji respons terhadap kenaikan dan penurunan set point. Simpan datanya.

Tabel 7: Performa implementasi kontrol kecepatan pada kit

T_s	**Metode**	**Kontrol**	t_d **(ms)**	t_r **(ms)**	M_p **(%)**	t_s **(ms)**	**Offset (%)**
T_{s1}	ZN	P					
		PI					
		PID					
	CC	P					
		PI					
		PID					
T_{s2}	ZN	P					
		PI					
		PID					
	CC	P					
		PI					
		PID					

4.4 Percobaan 4: Penalaan Halus

Penalaan halus dilakukan pada pengontrol PI dan PID untuk kedua waktu cacah. Pengontrol P tidak ditala halus karena selalu menghasilkan offset. Target penalaan adalah respons yang cepat, tanpa offset, dengan *overshoot* yang tidak berlebihan.

1. Mulai dari parameter metode yang performanya lebih baik pada Percobaan 3. Ubah satu parameter setiap kali dengan berpedoman pada Tabel 3. Gunakan simulasi (bagian parameter kustom pada skrip) untuk memperkirakan arah perubahan sebelum mencobanya pada kit.
2. Catat setiap langkah penalaan beserta alasannya. Simpan respons terbaik untuk setiap pengontrol dan setiap waktu cacah, lalu lengkapi Tabel 8.
3. **Penukaran parameter**: gunakan parameter terbaik T_{s1} pada waktu cacah T_{s2}, dan sebaliknya. Simpan responsnya dan lengkapi Tabel 9.

Tabel 8: Performa pengontrolan setelah penalaan halus

T_s	**Kontrol**	K_p	K_i	K_d	t_d	t_r	M_p	t_s	e_{ss}
T_{s1}	PI			–					
	PID								
T_{s2}	PI			–					
	PID								

Tabel 9: Performa pengontrolan dengan parameter ditukar

Dijalankan	**Parameter**	**Kontrol**	t_d **(ms)**	t_r **(ms)**	M_p **(%)**	t_s **(ms)**	**Offset (%)**
T_{s1}	T_{s2}	PI					
		PID					
T_{s2}	T_{s1}	PI					
		PID					

5 Laporan dan Analisis

5.1 Ketentuan Laporan

Laporan dikumpulkan per kelompok melalui Edunex paling lambat dua minggu setelah praktikum. Nilai tambahan diberikan bila laporan dikumpulkan dalam satu minggu. Laporan berupa file `.pdf` yang dilampiri *spreadsheet* pengolahan data, data mentah (`.txt`), program Arduino, dan skrip MATLAB, digabung dalam satu file `.zip`. Laporan minimal memiliki susunan berikut: latar belakang, tujuan, alat, prosedur, analisis, kesimpulan, kontribusi, dan referensi.

Latar belakang, tujuan, alat, dan prosedur tidak boleh sama persis dengan modul ini. Analisis disusun dengan alur yang runtut dengan menjawab pertanyaan pada Bagian 5.2. Perhatikan penggunaan angka penting. Bagian kontribusi berisi Tabel 10.

Tabel 10: Kontribusi anggota kelompok

No	**NIM**	**Nama Lengkap**	**Kontribusi**
1			
2			
3			
4			
5			

5.2 Analisis

Parameter kontrol dan simulasi.

1. Tunjukkan respons kalang terbuka hasil simulasi dan hasil kit dalam satu grafik. Apakah keduanya mirip? Bila berbeda signifikan, apa penyebabnya, dan apa artinya bagi model Modul 1?
2. Berikan komentar tentang nilai parameter kontrol antara metode Ziegler–Nichols dan Cohen–Coon. Mengapa nilai K_p, K_i, dan K_d sama untuk kedua waktu cacah?
3. Jelaskan cara mengidentifikasi setiap parameter performa dari data respons.
4. Berdasarkan simulasi, pengontrol dan metode mana yang memberikan performa terbaik untuk setiap waktu cacah? Jelaskan.

Implementasi kontrol kecepatan.

5. Jelaskan fungsi `control()` pada tab `5_control.ino` dan tuliskan dalam persamaan matematika. Jelaskan pula cara kerja *anti-windup* yang digunakan.
6. Tunjukkan respons kalang tertutup hasil simulasi dan hasil kit dalam satu grafik untuk setiap pengontrol. Apakah performanya mirip? Jelaskan penyebab perbedaannya.
7. Bandingkan performa antar struktur kontrol (P, PI, PID), antar metode (ZN, CC), dan antar waktu cacah. Pengontrol mana yang terbaik pada kit? Apakah sama dengan hasil simulasi?
8. Mengapa pengontrol P tidak dapat mencapai set point? Bandingkan offset hasil percobaan dengan Persamaan (5).
9. Bagaimana pengaruh waktu cacah terhadap performa? Kaitkan dengan Persamaan (4).
10. Tunjukkan respons terhadap urutan set point `{0, 50, 80, 20, 0}`. Apakah performa sama untuk set point naik dan turun, serta untuk set point kecil dan besar? Kaitkan dengan ketidaklinieran yang ditemukan pada Modul 1.

Penalaan halus.

11. Jelaskan tahapan penalaan halus yang dilakukan beserta alasan setiap perubahan parameter.
12. Tunjukkan respons sebelum dan sesudah penalaan halus dalam satu grafik. Apakah penalaan halus menghasilkan performa yang jauh lebih baik?
13. Tunjukkan respons dengan parameter ditukar. Apakah performanya lebih baik atau lebih buruk? Apa yang dapat disimpulkan tentang ketergantungan parameter terhadap waktu cacah?
14. Tentukan satu parameter kontrol terbaik untuk setiap waktu cacah dan lengkapi Tabel 11.

Tabel 11: Parameter kontrol dan performa pengontrolan terbaik

T_s	**Kontrol**	K_p	K_i	K_d	t_d	t_r	M_p	t_s	e_{ss}
T_{s1}									
T_{s2}									

6 Tugas Tambahan

Tugas tambahan bersifat opsional dan merupakan persiapan Modul 3.

1. Ubah aksi derivatif agar dihitung dari pengukuran ($-K_d\,\Delta y/T_s$), bukan dari error. Bandingkan respons terhadap perubahan set point (*derivative kick*).
2. Turunkan model posisi dari model kecepatan dengan memperhatikan perbandingan roda gigi motor dan potensiometer (16:100). Asumsikan kecepatan maksimum hasil Modul 1 setara dengan 6400 rpm.

3. Bagaimana respons posisi kalang terbuka bila diberi masukan step? Bagaimana bila diberi masukan impuls? Apakah pengontrol I diperlukan untuk kontrol posisi?
4. Siapkan program utama untuk mengontrol posisi. Bagian mana dari fungsi kontrol dan kompensasi yang perlu diubah?

Daftar Pustaka

[1] D. E. Seborg, T. F. Edgar, D. A. Mellichamp, dan F. J. Doyle III, *Process Dynamics and Control*, edisi ke-4. Hoboken, NJ: Wiley, 2016.

[2] K. J. Åström dan B. Wittenmark, *Computer-Controlled Systems: Theory and Design*, edisi ke-3. Englewood Cliffs, NJ: Prentice Hall, 1997.

A Kode Program Pendukung

Tab `5_control.ino` versi Modul 2 menggantikan versi sebelumnya. Program utama `8_main_kontrol_kecepatan` menggantikan isi tab `9_main.ino`. Tab lain tetap sama seperti pada Modul 1.

Kode 1: Tab `5_control.ino` (versi Modul 2)

```
// ============================================================================
// Tab kontrol: PID diskrit dengan pembatas keluaran dan anti-windup
//
// Satuan yang digunakan:
//   Error         : persen (%)
//   Ts            : milidetik (ms)
//   Kp            : -        (persen masukan per persen error)
//   Ki = Kp / Ti  : 1/ms
//   Kd = Kp * Td  : ms
// Nilai Kp, Ki, Kd, OutputMin, dan OutputMax diatur pada setup() program utama.
// ============================================================================

float Kp = 1.0, Ki = 0.0, Kd = 0.0;
float OutputMin = 0.0, OutputMax = 100.0;
float LastError = 0.0, AccumulationError = 0.0;
bool FirstSample = true;

void control_reset() {
  LastError = 0.0;
  AccumulationError = 0.0;
  FirstSample = true;
}

float control(float Error, unsigned int Ts) {
  float DeltaError = 0.0;
  if (!FirstSample) DeltaError = (Error - LastError) / Ts;
  FirstSample = false;

  float Output = Kp * Error + Ki * (AccumulationError + Error * Ts) + Kd *
    DeltaError;

  // Anti-windup (integrasi bersyarat): error hanya diakumulasi bila keluaran
  // tidak saturasi, atau bila error membantu keluaran keluar dari saturasi.
  if (Output > OutputMax) {
    Output = OutputMax;
    if (Error < 0) AccumulationError += Error * Ts;
  }
  else if (Output < OutputMin) {
    Output = OutputMin;
    if (Error > 0) AccumulationError += Error * Ts;
  }
  else {
    AccumulationError += Error * Ts;
  }

  LastError = Error;
  return Output;
}
```

Kode 2: Program utama `8_main_kontrol_kecepatan`

```
// ============================================================================
// Program utama Modul 2: kontrol kecepatan motor DC
```

```
// Ganti isi tab 9_main.ino dengan program ini.
//
// Set point mengikuti urutan SetPointSequence[]; setiap nilai ditahan selama
// HoldTime. Bila ClosedLoop = false, set point langsung diberikan sebagai
// masukan motor (kalang terbuka) sebagai pembanding.
// Kecepatan hanya dikontrol untuk satu arah putar karena bacaan tachometer
// selalu positif.
// ============================================================================

unsigned long Time, StartTime, LastTime, StepStart;
unsigned int TimeSampling = 10, DeltaTime;   // waktu cacah (ms)
unsigned int HoldTime = 3000;                // lama tiap set point (ms)
bool ClosedLoop = true;                      // false: kalang terbuka

float SetPointSequence[] = {0, 60, 0};       // contoh lain: {0, 50, 80, 20, 0}
const byte SequenceLength = sizeof(SetPointSequence) / sizeof(SetPointSequence[0]);

byte Index = 0;
float SetPoint = 0.0, Speed = 0.0, ManipulatedVariable = 0.0, Motor = 0.0;
bool Finished = false;

void setup() {
  general_setup();
  sensor_setup();
  actuator_setup();

  // Parameter pengontrol (Ki dalam 1/ms, Kd dalam ms)
  Kp = 1.0;
  Ki = 0.0;
  Kd = 0.0;
  OutputMin = 0.0;
  OutputMax = 100.0;
  control_reset();

  Serial.println("Time\tDelta\tSP\tKec\tMV\tMotor");
  StartTime = millis();
  LastTime = StartTime;
  StepStart = StartTime;
}

void loop() {
  Time = millis();
  if (Time - LastTime >= TimeSampling) {
    DeltaTime = Time - LastTime;
    LastTime = Time;

    if (Finished) {
      run_motor(0.0);
      return;
    }

    SetPoint = SetPointSequence[Index];
    Speed = smooth_velocity_percentage();

    if (SetPoint <= 0.0) {
      ManipulatedVariable = 0.0;
      control_reset();
    }
    else if (ClosedLoop) {
      ManipulatedVariable = control(SetPoint - Speed, TimeSampling);
    }
```

```
    else {
      ManipulatedVariable = SetPoint;
    }

    Motor = run_motor(compensation(ManipulatedVariable));

    Serial.print(Time - StartTime);
    Serial.print("\t");
    Serial.print(DeltaTime);
    Serial.print("\t");
    Serial.print(SetPoint);
    Serial.print("\t");
    Serial.print(Speed);
    Serial.print("\t");
    Serial.print(ManipulatedVariable);
    Serial.print("\t");
    Serial.println(Motor);

    if (Time - StepStart >= HoldTime) {
      StepStart = Time;
      Index++;
      if (Index >= SequenceLength) {
        Finished = true;
        Serial.println("# selesai");
      }
    }
  }
}
```

Kode 3: Skrip MATLAB `simulasi_kontrol_kecepatan.m`

```
%% simulasi_kontrol_kecepatan.m
% Simulasi kontrol kecepatan motor DC (Modul 2).
% Plant FOPDT disimulasikan dengan langkah 1 ms, sedangkan pengontrol PID
% diskrit dijalankan setiap waktu cacah Ts dengan pembatas keluaran dan
% anti-windup yang sama dengan tab 5_control.ino.
%
% Skrip ini:
%   1. menghitung parameter Ziegler-Nichols dan Cohen-Coon,
%   2. mensimulasikan P, PI, PID untuk dua waktu cacah,
%   3. menghitung parameter performa (delay time, rise time, overshoot,
%      settling time, offset),
%   4. mensimulasikan parameter kustom hasil penalaan halus.

clc; clear; close all;

%% 1. Model sistem dari Modul 1 (waktu dalam ms)
K     = 0.85;      % penguatan
Tau   = 60;        % konstanta waktu (ms)
Theta = 30;        % waktu tunda (ms)
Ts1   = 10;        % waktu cacah pertama (ms)
Ts2   = 3*Ts1;     % waktu cacah kedua (ms)

SetPoint = 60;     % set point (%), sekitar 75% kecepatan maksimum
TSim     = 2000;   % durasi simulasi (ms)
UMin = 0; UMax = 100;

%% 2. Parameter Ziegler-Nichols dan Cohen-Coon
r = Theta / Tau;
% baris: P, PI, PID; kolom: Kp, Ti, Td (Ti = Inf berarti tanpa aksi integral)
ZN = [ Tau/(K*Theta),        Inf,                 0;
```

```
      0.9*Tau/(K*Theta),    3.33*Theta,          0;
      1.2*Tau/(K*Theta),    2*Theta,             0.5*Theta ];
CC = [ (Tau/(K*Theta))*(1 + r/3),     Inf,                              0;
      (Tau/(K*Theta))*(0.9 + r/12),  Theta*(30 + 3*r)/(9 + 20*r),      0;
      (Tau/(K*Theta))*(4/3 + r/4),   Theta*(32 + 6*r)/(13 + 8*r),      4*Theta/(11 + 2*r) ];
NamaKontrol = {'P', 'PI', 'PID'};
NamaMetode  = {'ZN', 'CC'};
Tabel = {ZN, CC};

fprintf('Parameter kontrol (Ti, Td dalam ms; Ki dalam 1/ms; Kd dalam ms)\n');
for m = 1:2
    for j = 1:3
        [Kp, Ki, Kd] = konversi(Tabel{m}(j, :));
        fprintf('  %s %-3s : Kp = %7.3f  Ti = %7.1f  Td = %6.1f  Ki = %8.5f  Kd = %7.2f\n', ...
            NamaMetode{m}, NamaKontrol{j}, Tabel{m}(j,1), Tabel{m}(j,2), Tabel{m}(j,3), Ki, Kd);
    end
end

%% 3. Simulasi kalang terbuka (pembanding data kit)
[tOL, yOL] = simulasi(K, Tau, Theta, Ts1, SetPoint, TSim, 0, 0, 0, UMin, UMax, false);
figure('Name', 'Kalang terbuka');
plot(tOL, yOL, 'LineWidth', 1.2); grid on;
xlabel('Waktu (ms)'); ylabel('Kecepatan (%)');
title(sprintf('Kalang terbuka, masukan %g%%', SetPoint));

%% 4. Simulasi kalang tertutup untuk semua kombinasi
fprintf('\nPerforma (td, tr, ts dalam ms; OS dan offset dalam %%)\n');
for Ts = [Ts1 Ts2]
    figure('Name', sprintf('Kalang tertutup Ts = %g ms', Ts));
    for m = 1:2
        subplot(1, 2, m); hold on; grid on;
        for j = 1:3
            [Kp, Ki, Kd] = konversi(Tabel{m}(j, :));
            [t, y] = simulasi(K, Tau, Theta, Ts, SetPoint, TSim, Kp, Ki, Kd, UMin, UMax, true);
            P = kinerja(t, y, SetPoint);
            fprintf('  Ts=%2g %s %-3s : td=%6.1f tr=%6.1f OS=%6.1f ts=%7.1f offset=%6.2f\n', ...
                Ts, NamaMetode{m}, NamaKontrol{j}, P);
            stairs(t, y, 'DisplayName', NamaKontrol{j});
        end
        yline(SetPoint, 'k--', 'HandleVisibility', 'off');
        xlabel('Waktu (ms)'); ylabel('Kecepatan (%)');
        title(sprintf('%s, Ts = %g ms', NamaMetode{m}, Ts)); legend('Location', 'southeast');
    end
end

%% 5. Parameter kustom (penalaan halus)
KpKustom = 2.0; KiKustom = 0.02; KdKustom = 0; TsKustom = Ts1;
[t, y] = simulasi(K, Tau, Theta, TsKustom, SetPoint, TSim, KpKustom, KiKustom, KdKustom, UMin, UMax, true);
P = kinerja(t, y, SetPoint);
fprintf('\nKustom: td=%6.1f tr=%6.1f OS=%6.1f ts=%7.1f offset=%6.2f\n', P);
figure('Name', 'Kustom'); stairs(t, y); grid on; yline(SetPoint, 'k--');
xlabel('Waktu (ms)'); ylabel('Kecepatan (%)'); title('Parameter kustom');

```

```
%% Fungsi bantu
function [Kp, Ki, Kd] = konversi(Baris)
    Kp = Baris(1);
    if isinf(Baris(2)), Ki = 0; else, Ki = Kp / Baris(2); end
    Kd = Kp * Baris(3);
end

function [tk, yk] = simulasi(K, Tau, Theta, Ts, SP, TSim, Kp, Ki, Kd, UMin, UMax,
    Tertutup)
    % Plant FOPDT (langkah 1 ms) dengan pengontrol PID diskrit setiap Ts
    N = TSim; d = round(Theta);
    ubuf = zeros(N + d + 1, 1);
    v = 0; u = 0; Acc = 0; LastE = 0; First = true;
    nk = floor((N - 1) / Ts) + 1; tk = zeros(nk, 1); yk = zeros(nk, 1); i = 0;
    for k = 0:N-1
        if mod(k, Ts) == 0
            if Tertutup
                e = SP - v;
                de = 0; if ~First, de = (e - LastE) / Ts; end
                First = false;
                u = Kp*e + Ki*(Acc + e*Ts) + Kd*de;
                if u > UMax
                    u = UMax; if e < 0, Acc = Acc + e*Ts; end
                elseif u < UMin
                    u = UMin; if e > 0, Acc = Acc + e*Ts; end
                else
                    Acc = Acc + e*Ts;
                end
                LastE = e;
            else
                u = SP;
            end
            i = i + 1; tk(i) = k; yk(i) = v;
        end
        ubuf(k + 1 + d) = u;                % masukan tertunda Theta
        v = v + (K*ubuf(k + 1) - v) / Tau;  % Euler, langkah 1 ms
    end
end

function P = kinerja(t, y, SP)
    % [delay time, rise time 10-90%, overshoot, settling time 2%, offset]
    yf = mean(y(end - max(1, round(0.1*numel(y))) + 1:end));
    td = waktu_pertama(t, y >= 0.5*yf);
    tr = waktu_pertama(t, y >= 0.9*yf) - waktu_pertama(t, y >= 0.1*yf);
    OS = max(0, (max(y) - yf) / yf * 100);
    luar = find(abs(y - yf) > 0.02*abs(yf));
    if isempty(luar), ts = t(1); elseif luar(end) < numel(t), ts = t(luar(end) + 1);
     else, ts = NaN; end
    P = [td, tr, OS, ts, SP - yf];
end

function tw = waktu_pertama(t, Kondisi)
    k = find(Kondisi, 1);
    if isempty(k), tw = NaN; else, tw = t(k); end
end
```